

ACX Platform (HMS): Hospitality Management System — Quick Reference

Inspironics Corporation

Program Documentation, ACX Platform HMS • Baseline Issue (API v0.3.0) • September 25, 2026

Abstract—This card condenses the operation of ACX HMS — the Inspironics hospitality management platform — into lookup tables for administrators, duty managers, front-desk and housekeeping supervisors and operators. It covers the API conventions and endpoint set, the route and module map, the task-to-permission lookup, the seeded role matrix, the stay and room state rules, the status vocabularies, the error taxonomy, and the operator configuration and command set. It is a recall aid, not an introduction; the User Manual [1] teaches the platform and the Troubleshooting Guide [2] diagnoses it.

Index Terms—quick reference, permission model, access control, stay lifecycle, room status, error taxonomy, operations, configuration management.

I. INTRODUCTION

Full manuals serve learning; quick references serve recall. Everything on this card is drawn from the HMS baseline build and its design records [3, 4, 5] and is arranged for scanning rather than reading. Six rules govern every table that follows, and are stated once here so that they need not be repeated in each:

1. Only admin and manager roles can sign in to HMS Web; every other role is refused by platform.
2. Every route and every endpoint requires read on its module; every change requires write.
3. Authorization is decided by the server from the database on every request, never from the client.
4. Rooms change status only through booking, check-in, check-out, reassign and cancel actions while a live stay holds them.
5. Nothing is hard-deleted: records are deactivated, cancelled or soft-deleted (`status = 0`).
6. A figure that cannot be computed is reported as unavailable, never shown as zero.

II. ACCESS, SESSION AND API CONVENTIONS

All HTTP interfaces are served under `/api/v1/`; liveness is `GET /health` outside the prefix. The OpenAPI description is generated from the implementation and served at `/docs`, `/redoc` and `/openapi.json`. List endpoints return one pagination envelope and every failure returns one error envelope:

```
{ "items": [ ... ],
  "page": 1, "page_size": 20,
  "total": 27 }

{ "error": { "code": "conflict",
             "message": "...",
             "detail": [ ... ] } }
```

page starts at 1; page_size defaults to 20 with a maximum of 100 (larger values return 422). Sign-in is `POST /auth/login` with username and password; the response carries a bearer access_token (HS256, claims sub, typ, iat, exp) and expires_in. Lifetime defaults to 60 minutes and there is no refresh token. Table I lists the endpoint set by area.

TABLE I
BASELINE ENDPOINT SET

Area	Endpoints
Health	GET /health (unprefixed); GET /health/db
Auth	POST /auth/login; GET /auth/me
People	GET POST /users; PATCH /users/{id}; POST /users/{id}/{deactivate, reactivate, password}; /departments; /job-functions
Roles	GET POST /roles; PATCH /roles/{id}; GET PUT /roles/{id}/permissions; GET /modules; GET /permissions
Facility	GET /facilities, /properties, /buildings, /floors; GET POST PATCH /rooms; /amenity-types; /packages; /features
Stays	GET POST /stays; PATCH /stays/{id}; POST /stays/{id}/{check-in, check-out, extend, status, cancel}; POST /stays/{id}/documents/approval
Allocation	POST /stays/{id}/room-allocations; PATCH DELETE /room-allocations/{id}; POST /stays/{id}/occupants; DELETE /stay-occupants/{id}
Occupancy	GET /occupancy; PATCH /occupancy/{id}; PUT /occupancy/{id}/conditions; GET /amenity-statuses; GET /amenity-conditions
Services	GET POST /service-requests; PATCH /service-requests/{id}; PUT .../{id}/items; POST .../{id}/cancel; /service-types; /service-categories; /service-items
Planning	GET POST /maintenance-requests; PATCH DELETE .../{id}; POST .../{id}/cancel; GET POST PATCH DELETE /job-orders
Devices	GET POST /devices; POST /devices/{id}/{commission, decommission, maintenance}; /alerts; /incidents; /value-alerts; /limit-configs
Marketing	GET POST PATCH /offers, /events, /holidays
Reporting	GET /reports; GET /reports/{key}; GET /reports/{key}/export.xlsx; /energy-stats; /daily-data-points

Area	Endpoints
Activity	GET /notifications; GET /activities; GET /notification-templates

III. NAVIGATION AND PERMISSION MAPPING

Table II maps each route to the module whose read grant reveals it. Screen visibility and API permission are the same grant with one exception worth remembering: /tickets is revealed by tickets, but the service-request API behind it is governed by service_tracking. A role holding the first without the second sees the screen and is refused its data.

TABLE II
ROUTE TO GOVERNING MODULE

Route	Module (read to view, write to act)
/HMS	dashboard (read only)
/occupancy	occupancy; check-out and reassign need bookings:write
/bookings	bookings
/services/tracking	service_tracking
/services/planning	service_planning
/tickets	tickets (data via service_tracking)
/config/facility	facility_management
/config/user-roles	user_roles
/config/services-setup	service_setup
/config/employees	employees
/config/job-order	job_order
/config/limit-alert	caleido_network
/offers, /holidays, /events	offers, holidays, events
/reports/{key}	reports (read only)
/key-settings	default_key

IV. COMMON TASKS AND REQUIRED PERMISSIONS

Table III extracts the entries with paired or non-obvious requirements. Several actions touch two modules; holding one alone produces a refused action rather than a hidden control.

TABLE III
SELECTED TASKS AND REQUIRED PERMISSIONS

Task	Permission(s)
Create a booking	bookings:write (guest record created in the same flow)
Check in / check out	bookings:write + stay state permits it
Check out from Occupancy	occupancy:read + bookings:write
Reassign a room	bookings:write + an existing stay + a free room
Mark room conditions	occupancy:write
View room environment tab	caleido_network:read or reports:read
Raise or assign a ticket	service_tracking:write
Schedule planned service	service_planning:write; assignees must be staff
Acknowledge an incident	caleido_network:write
Set a user's password	user_roles:write
Edit a role's module matrix	user_roles:write; write needs read
Pick building and floor	facility_management:read
Export a report	reports:read
Generate an invoice	not connected in this baseline

Two facts deserve emphasis because they are asked about repeatedly. A Duty Manager holds no facility_management grant, so building and floor lists are refused for that role by design. And staff roles (Front Desk, Housekeeping, Technician) hold grants that apply on the mobile platform; they cannot sign in to HMS Web regardless of those grants. Table IV gives the seeded matrix.

TABLE IV
SEEDED ROLE MATRIX (R READ, W READ+WRITE)

Module	Adm	DM	FD	HK	Tech
dashboard	R	R	R	R	R
occupancy	W	W	W	R	—
bookings	W	W	W	—	—
service_tracking	W	W	R	W	W
service_planning	W	W	—	W	—
tickets	W	W	W	—	—
job_order	W	W	—	—	W
employees	W	W	—	—	—
offers / events / holidays	W	W	—	—	—
caleido_network	W	R	—	—	W
reports	R	R	—	—	—
facility_management	W	—	—	—	—
user_roles / service_setup	W	—	—	—	—
default_key	W	—	W	—	—
firmware_management	W	—	—	—	W

V. STAY LIFECYCLE AND ROOM STATUS

Stay statuses: pending, active, checkout pending, checkout accepted, checkout rejected, checked out, cancelled. The first five are *live* and hold their rooms. Room statuses: 0 Available, 1 Occupied, 2 Unavailable, 3 Allotted. Room conditions, independent of status: Dirty (shown as Need Maintenance), Low battery, Under maintenance, Sanitation, Ready for occupant. Fig. 1 shows how stay actions drive room status; Table V gives the preconditions.

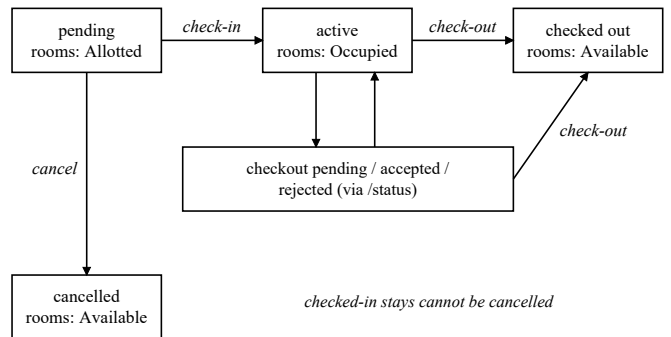


Fig. 1. Stay states and the room status each implies. Room conditions are not changed by any stay action.

TABLE V
STAY ACTIONS, PRECONDITIONS AND EFFECTS

Action	Requires	Effect
Create	check-out after check-in; free rooms	pending; rooms Allotted
Check in	not in, out or cancelled; ≥ 1 room	active; rooms Occupied; time stamped
Check out	checked in; not already out	checked out; rooms Available; initiator recorded
Extend	not checked out; later time	expected check-out moved
Set status	live stay; not out/cancelled	checkout approval states only
Cancel	not checked in; not cancelled	cancelled; rooms Available
Reassign	live stay; different free room	old room Available; new room Occupied or Allotted
Release room	stay not in house	allocation removed

Reference formats: stays STY-YYYY-NNNN, service requests SR-YYYY-NNNN, job orders JO-YYYY-NNNN, invoices INV-YYYY-NNNN.

VI. STATUS VOCABULARIES

TABLE VI
OPERATIONAL STATUS SETS

Object	States
Service request	1 Pending, 2 Assigned, 3 Partially completed, 4 Completed, 5 Canceled
Planned service	same set; types scheduled, planned, disinfection; weekly recurrence
Job order	pending, completed; work installation, replacement, troubleshoot
Incident	1 Unread, 2 Read, 3 Assigned, 4 Resolved; leaving Resolved logs Reopened
Device config	configured, bad_configuration, commissioned, decommissioned, under_maintenance, missing
Device health	Active, Inactive
Alert severity	warning, critical
Documents	stay: pending, approved; document: pending, approved, rejected

The seven service types are Room Service, Travel Desk, Business Center, Food Order, Facility Maintenance Service, Health & Fitness and Sanitation Maintenance Service. The nine report keys are occupancy, employee, room-status, booking, ticket, housekeeping, sanitization, alert and energy.

VII. ERROR CODES AT A GLANCE

Table VII groups the taxonomy by HTTP status with the messages most often seen. The code field is the stable vocabulary; messages may name the record involved.

TABLE VII
ERROR TAXONOMY BY STATUS

HTTP	Code and typical message
400	bad_request
401	unauthorized — Not authenticated.; Invalid username or password.
403	forbidden — Role does not grant {read write} access to the '{module}' module.; This account belongs to the {platform} application and cannot sign in to HMS Web.
404	not_found — {Label} {id} does not exist.; No report named '{key}'.
405	method_not_allowed
409	conflict — That room is already allocated to stay {ref}.; Stay {ref} still holds this room; check it out first.; This stay has not been checked in.; {Label} '{v}' already exists.
422	validation_error — Request validation failed. (with detail[].loc, msg); Allocate a room before checking this stay in.; Expected check-out must be after expected check-in.
500	internal_error — Internal server error.
503	service_unavailable — Database is unavailable.

Two conventions depart from the obvious reading. A 409 means the request was well-formed but the *current state* forbids it — resolve the state, not the request. And a 422 on a list filter usually means a value the column's enumeration does not accept, reported as *A query value is not valid for its column*.

VIII. OPERATOR CHEAT SHEET

A. Local Stack

Three processes: PostgreSQL 16 or later on port 5432 (database hms_db), the API under Uvicorn on port 8000, and the Vite client on port 8080, which proxies /api/v1 to the API so no CORS set-up is needed locally. No container, cache, queue or scheduler is required.

B. Configuration

Backend variables are read from backend/.env (template .env.example); client variables from frontend/.env.local. A .env holding credentials is never committed.

TABLE VIII
ENVIRONMENT CONFIGURATION

Group	Variables
Database	POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB, POSTGRES_SCHEMA, DATABASE_URL (overrides)
Application	APP_NAME, APP_ENV, DEBUG, API_V1_PREFIX, CORS_ORIGINS
Identity	JWT_SECRET_KEY, JWT_ALGORITHM (HS256), JWT_ACCESS_TOKEN_EXPIRE_MINUTES (60)
Client	VITE_API_BASE_URL (/api/v1), VITE_API_PROXY_TARGET (http://127.0.0.1:8000)

C. Commands

TABLE IX
BUILD, RUN AND VERIFY

Purpose	Command
Schema	<code>alembic upgrade head</code>
Seed demo data	<code>python -m seeds.run_seed(--report to inspect)</code>
Run API	<code>uvicorn app.main:app --reload</code>
Backend tests	<code>python -m pytest (fresh seed required)</code>
Verify KPIs	<code>python -m scripts.verify_dashboard_kpis</code>
Run client	<code>npm ci then npm run dev</code>
Client checks	<code>npm run lint; npx tsc -b --noEmit; npm run build</code>

D. Seed Data and First Sign-In

The seed creates facility *Ikanos Grand Chennai*, Tower A with floors 1–3, rooms 101–308, four packages, six stays, fourteen devices and thirteen users. **No seeded account can sign in:** every seeded password hash is a placeholder. Set a bcrypt hash for an admin or manager user (for example `arjun.menon`) before first use.

E. Production Checklist

Set `APP_ENV` to a non-development value and a unique `JWT_SECRET_KEY` — the service refuses to sign tokens with the default secret outside development. Set `CORS_ORIGINS` to the served client origin and `VITE_API_BASE_URL` to the absolute API URL, and turn `DEBUG` off.

IX. CONCLUSION

Held beside a screen, this card should answer three questions without recourse to the manual: what module does this action need, what state must this stay be in, and what does this error mean. Where it cannot — invoicing, payments, bulk upload, default keys, notification delivery and multi-facility operation — the baseline has not yet connected the capability, and the card says so rather than supplying a plausible answer.

REFERENCES

- [1] Inspironics Corporation, “INS-ACX-HMS-UM-1.0 — ACX HMS User Manual,” Sep. 2026.
- [2] Inspironics Corporation, “INS-ACX-HMS-TS-1.0 — ACX HMS Troubleshooting Guide,” Sep. 2026.
- [3] Inspironics Corporation, “FINAL_HMS_DATABASE_BLUEPRINT — HMS Database Blueprint of Record,” backend/docs, 2026.
- [4] Inspironics Corporation, “PHASE2_4_AUTHENTICATION — JWT Authentication and Module Permissions,” backend/docs, 2026.
- [5] Inspironics Corporation, “PHASE1_8_SEED_DATA — Demonstration Seed Data,” backend/docs, 2026.